

Clase 158 — Emulación y unpacking automatizado

Parte: **6 — Análisis de malware** · Fuente: *Practical Malware Analysis* y documentación de Unicorn/qiling 🕒 Duración estimada: **130 min** · Nivel: **Experto**

🎯 Objetivo

Escalar el análisis con **emulación** de CPU y automatización del unpacking y la extracción de configuración. El alumno aprenderá a usar frameworks de emulación (Unicorn, Qiling) y de análisis programático (Speakeasy, dumpulator) para desempacar sin depurar a mano, resolver API hashing, ejecutar rutinas de descifrado aisladas y construir extractores de config reutilizables.

📄 Resultados de aprendizaje

Al finalizar, el alumno podrá:

1. **Explicar** cuándo la emulación supera a la depuración manual.
2. **Emular** funciones aisladas (descifrado, API hashing) con Unicorn/Qiling.
3. **Automatizar** el unpacking y el dump con emuladores o hooks.
4. **Construir** un extractor de configuración para una familia.
5. **Integrar** la automatización en un pipeline de triaje.

📖 Temas

#	Tema	Por qué importa
1	Emulación vs ejecución vs depuración	Control y seguridad
2	Unicorn Engine (emulación de CPU)	Ejecuta código sin SO real
3	Qiling (emulación con SO)	Emula syscalls y librerías
4	Speakeasy / dumpulator	Emulación orientada a malware
5	Resolución de API hashing por emulación	Recupera nombres de API
6	Extractores de configuración	Config a escala
7	Pipelines de triaje	Automatización operativa

📖 Definiciones y características

- **Emulación:** ejecutar instrucciones en un motor software, no en la CPU real. Característica clave: **control total y aislamiento**.
- **Unicorn Engine:** emulador de CPU multiarquitectura. Característica clave: **emula código, no el SO**.
- **Qiling:** framework de emulación con soporte de SO/syscalls. Característica clave: **ejecuta binarios enteros** de forma controlada.

- **Config extractor:** script que extrae la configuración de una familia. Característica clave: **repetible y a escala**.
- **API hashing:** resolver APIs por hash. Característica clave: **resoluble por emulación** del algoritmo.

Herramientas y preparación

- **Unicorn Engine** (+ Capstone/Keystone) y **Qiling Framework**.
- **Speakeasy** (Mandiant) y **dumpulator** para emulación de malware.
- **FLOSS/capa** para localizar rutinas objetivo.
- Python en la VM aislada para orquestar.

 **Nota ética y de seguridad:** aunque la emulación es más segura que ejecutar, trabaja en la VM aislada: los binarios siguen siendo maliciosos y algunos emuladores realizan syscalls reales. No permitas salida de red desde el emulador. Los extractores se usan para defensa.

Laboratorio guiado

1. Con `capa` /`FLOSS` localiza en una muestra la rutina de **descifrado de config** o de **API hashing** (dirección de inicio y fin).
2. Emula solo esa función con Unicorn: mapea memoria, carga los bytes del binario, fija registros/argumentos y ejecuta hasta la dirección final. Lee el buffer de salida (config descifrada).

```
python from unicorn import * from unicorn.x86_const import * mu = Uc(UC_ARCH_X86, UC_MODE_32)
mu.mem_map(0x400000, 0x100000) mu.mem_write(0x401000, code_bytes) mu.reg_write(UC_X86_REG_ESP,
0x4FF000) mu.emu_start(0x401000, 0x401050) print(mu.mem_read(0x402000, 64)) # config
descifrada
```

3. Para API hashing, emula el algoritmo de hash y precomputa los hashes de APIs comunes; construye un mapa hash→nombre y anota el desensamblado.
4. Usa **Qiling** o **Speakeasy** para emular la muestra completa y capturar las llamadas a APIs (`VirtualAlloc`, `WriteProcessMemory`) que revelan el payload desempacado; vuelca la región.
5. Empaqueta lo anterior en un **extractor de config**: recibe una muestra de la familia y devuelve dominios/claves/intervalos en JSON.
6. Prueba el extractor contra varias muestras de la familia y mide su tasa de éxito.

Ejercicios

1. Explica cuándo prefieres emular una función en vez de depurar la muestra.
2. Emula una rutina XOR con Unicorn y recupera su salida.
3. Resuelve API hashing precomputando hashes de APIs comunes.
4. Usa Qiling/Speakeasy para capturar las APIs de un unpacker.
5. Escribe un extractor de config que devuelva JSON.
6. Integra el extractor en un mini-pipeline de triaje por lotes.

Reto verificable

Entrega un **extractor de configuración** para una familia que, por emulación, recupere la config (dominios/clave/intervalo) de al menos 3 muestras distintas de esa familia, con salida en JSON. **Criterio de aceptación:** el script corre sin depuración manual y produce la config correcta para 3+ muestras de la familia, verificable contra el análisis manual de una de ellas.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
La emulación falla en una API	Falta stub/hook de esa API; impleméntalo o usa Qiling
Registros/pila mal inicializados	Emulas una función sin su contexto; fija args y ESP/RSP
Config sale corrupta	Rango de direcciones incorrecto; ajusta inicio/fin y base
Hashes no coinciden	Algoritmo de hash mal identificado; verifícalo en el código
Extractor solo sirve para 1 muestra	Codificaste offsets fijos; parametriza por patrones

Preguntas frecuentes

 **¿Emulación reemplaza al desensamblado?** No; lo complementa. Primero localizas la rutina con análisis estático y luego la emulas para obtener el resultado sin ejecutar todo el malware.

 **¿Es 100% segura la emulación?** Más segura que ejecutar, pero frameworks con soporte de SO pueden hacer syscalls reales; mantén el aislamiento.

 **¿Vale la pena automatizar un extractor?** Sí cuando una familia es prevalente: procesar cientos de muestras a mano no escala; un extractor entrega IOCs en segundos.

Referencias

- Unicorn Engine. <https://www.unicorn-engine.org>
- Qiling Framework. <https://qiling.io>
- Speakeasy (Mandiant). <https://github.com/mandiant/speakeasy>
- dumpulator. <https://github.com/mrexodia/dumpulator>

Siguiente clase

Clase 159 - Fileless malware y living-off-the-land